NTNU
Norwegian University of
Science and Technology

Formally Verified Reduction Proof for the Free XOR Garbling Scheme

Prateek Sharma

Submission date: 27 July 2026
Main supervisor: Tjerand Silde, NTNU
Co-supervisor: Russell W. F. Lai, Aalto University
Advisors: Chris Brzuska, Aalto University
Amirhosein Rajabi, Aalto University
Kirthivaasan Puniamurthy, Aalto University

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Garbled circuits are a fundamental cryptographic primitive used in secure two-party computation (2PC), enabling two parties to jointly compute a function without revealing their private inputs - only the output is revealed. The concept was introduced by Andrew Yao in 1986 [Yao86], where functions are expressed as Boolean circuits and each wire is labelled with encrypted wire keys. Early constructions required four ciphertexts per gate, resulting in significant communication overhead and limiting practical adoption.

Subsequent optimizations such as Point-and-Permute, Half-Gates and the Free XOR technique greatly reduced the number of ciphertexts needed per gate, as well as reducing the number of gates to be transmitted, leading to substantial performance improvements [BMR90; ZRE15; KS08]. These optimizations form the basis of many state-of-the-art secure computation frameworks.

Formal verification of cryptographic constructions is essential to ensure correctness and prevent subtle vulnerabilities arising from unchecked corner cases. Brzuska and Oechsner [BO23] developed a state-separating pen-and-paper reduction proof for Yao's original garbling scheme. State-separating proofs model protocol components as isolated "packages" with private state and oracle interfaces, allowing for structured security reductions. Their work established security by reducing the garbling scheme to the IND-CPA security of the encryption primitive used for gate tables.

Domino is an automated verification tool specifically designed for state-separating proofs. It has been used to reason about complex real-world protocols such as TLS [Raj25] and WPA2 [BDE+22]. Domino offers a high level of automation and readability because proof scripts resemble the pseudocode found in traditional cryptographic proofs.

Furthermore, the above mentioned proof for Yao's garbling scheme by Brzuska and Oechsner [BO23] was later formalized in Domino [BERW26]. It can serve as a good starting point for the thesis to observe how games may be converted from pen-and-paper proofs to Domino code.

The goal of this thesis is to formalize and mechanize a security reduction for the Half-Gates garbling scheme within the Domino framework. This includes:

- Developing a state-separating, pen-and-paper reduction proof for the Free XOR technique.
- Encoding this proof in Domino.

This work extends formal verification techniques to an optimized and widely deployed garbling construction and demonstrates Domino's applicability to modern cryptographic primitives.

Approved: 2026-03-09 – Tjerand Silde, NTNU (Main supervisor)

Abstract

Use some stuff from problem desc here

Preface

This thesis was submitted as part of the completion requirements for my Master of Science in Security and Cloud Computing double degree at Norwegian University of Science and Technology (NTNU) and Aalto University. It was supervised by Tjerand Silde at the Department of Information Security and Communication Technology (IIK) NTNU and Russell Lai at Aalto Univeristy, and was also advised by Chris Brzuska, Amirhosein Rajabi and Kirthivaasan Puniamurthi, also at Aalto University.

Acknowledgements

Acknowledgements are due

Contents

List of Figures

List of Acronyms

2PC	Two-party Computation
CCR	circular correlation-robust
CR	correlation-robust
FreeXOR	Free XOR
GC	Garbled Circuit
GRR	Garbled Row Reduction
GS	garbling scheme
HG	Half-Gates
LIN-RK-KDM	Linear, Related-Key, Key-Dependent-Message
LPN	Learning Parity with Noise
MPC	Multi-Party Computation
OT	Oblivious Transfer
OTP	one-time pad
P&P	Point and Permute
PPT	probabilistic, polynomial-time
PRF	Pseudo-Random Function
PRG	Pseudo-Random Generator
SDG	Sustainable Development Goal
SMT	Satisfiability Modulo Theory
SSP	state-separating proof
UN	United Nations

Chapter 1
Introduction

In Secure Multiparty Computation (also called just Multi-Party Computation) (MPC), two or more parties want to compute a common function using their individual inputs, but do not want to share these inputs with each other. Common use cases of MPC today include delegating computation to cloud servers [Urb24], or querying third party datasets that cannot be shared directly [LPR+21].

Since remote servers are often owned and operated by a separate entity, it is difficult to obtain the same security and privacy guarantees that a user's own machine can provide. Besides user requirements, cybersecurity regulations also require methods to secure the data being processed from the data processor, such as in the case of private healthcare data, or other forms of personal data [Eur16].

1.1 Motivation

Two-party Computation (2PC) is a special case of MPC in which only two parties are involved in the protocol. Often attributed to Andrew Yao's seminal work in 1986 [Yao86], the first approach for 2PC was to represent the function to be computed as a boolean circuit, and to use a Garbled Circuit (GC) to compute the output of the function, while maintaining the secrecy of the inputs of both parties. For more than a decade after this work, Garbled Circuits were considered to not be practical because of the high bandwidth and computation costs of garbling, and due to the fact that garbling an arbitrary function into an optimized boolean circuit is difficult. However, there has been work in the field that has led to optimizations that allow them to be somewhat usable in generic 2PC systems like Fairplay [MNPS04], and in niche applications such as smart contracts [HYLL24].

1.1.1 Garbling Schemes

Yao's version of Garbled Circuits used four ciphertext computations and transmissions per gate in the circuit. It relied on the standard assumption of existence of secure

Pseudo-Random Functions (PRFs) that could be used to build a secure encryption scheme. Beaver et al. published the next well-known optimization to garbled circuits, often called Point and Permute (P&P) (we discuss it in more detail in Section 2.3.1), which reduces the computational cost of evaluation of the circuit by up to 75% of Yao's construction, and uses a Pseudo-Random Generator (PRG) security assumption[BMR90]. We observe that while there are improvements, the core assumptions vary. This is a pattern that continues with Naor et al.'s Garbled Row Reduction-3 scheme (Section 2.3.2), which offers a flat 25% reduction in bandwidth cost, but returns to a PRF security assumption [NPS99]. The next improvement - the Free XOR scheme by Kolesnikov and Schneider can garble an XOR gate completely free of cost (Section 2.3.3), which can be very efficient if circuits are optimized to maximize the number of XOR gates [KS08]. However, Free XOR relies on an entirely new assumption, 'correlation-robustness' (CR) of a hash function. Furthermore, in their paper, Free XOR was only proven in the stronger Random Oracle model, and CR was later found to be inadequate. Free XOR was later proven secure using other models such as Linear, Related-Key, Key-Dependent-Message (LIN-RK-KDM) security of encryption schemes [App16], and circular correlation-robustness (CCR) of hash functions [CKKZ12]. Both of these assumptions are weaker than the ideal Random Oracle model, but stronger than CR. The Half-Gates garbling scheme was designed under the CCR assumption, and leads to a 50% decrease in bandwidth use over Yao's scheme when garbling AND gates [ZRE15].

1.1.2 Formal Verification

Unlike sofware verification, proof verification is not concerned with whether an implementation faithfully realises a protocol, but whether the mathematical argument establishing the security of the proof is logically sound. These concerns are orthogonal: an implementation may be correct while relying on an incorrect security proof, just as a correct proof does not guarantee a secure implementation. Validating a large cryptographic proof however is a difficult task, mainly because reasoning about code by simply looking at it relies on intuition, and may hide away corner cases especially in larger proofs. Halevi argued that an alternative is to use computers to enhance the process of reasoning about code [Hal05]. Two main approaches exist for automated verification of code - symbolic and computational. Symbolic verification uses an idealized model of cryptography (specifically, the Dolev-Yao model [DY81]) where primitives are treated as perfect black boxes. An adversary is only allowed to use these black boxes in its operations, limiting the list of operations it can perform to just such functions and their defined properties. This allows symbolic verification to be faster and makes it easier to build formal verification tools. For example, in the symbolic model, an encryption function may be modelled such that the adversary can derive the message only if they know the key. In contrast, the computational model explicitly models these cryptographic primitives as polynomial time reductions to some underlying assumption or game. The amount of 'assumed code' is much smaller and the only limit to the adversary is that it must be a polynomial time algorithm. Computational verifiers reason about the security of cryptographic algorithms under given computational assumptions, as opposed to symbolic verifiers which assume that cryptographic primitives always satisfy their symbolic equations. However, this leads to proofs that are written for computational verifiers to be more complex. Examples of symbolic verification tools include ProVerif [Bla16] and Tamarin [BCDS22], while EasyCrypt [BDG+14], CryptoVerif [Bla23] and Domino [BERW] are examples of computational verification tools.

1.1.3 State Separating Proofs

The security of cryptographic algorithms is often reasoned about as 'games' in an ideal and real world. The games consist of 'oracles' which are basically procedures that expose certain functionality and either return an output or modify some state. An adversary interacts with the game in both worlds by calling its oracles and attempts to distinguish between the behavior of the two games. These games were initally expressed as turing machines with polynomial running time, but Bellare and Rogaway reasoned that pseudocode was also a valid medium to express such games, and was suitable to use in future automated verification systems that Halevi wrote about [BR06]. Going a step further from Bellare and Rogaway's code-based game-playing, Brzuska et al. proposed state-separation, which would introduce the concept of games composed of packages with internal states which were modifiable by the oracles contained in those packages [BDF+18]. State-separating proofs (SSPs) are attractive because they allow cryptgraphers to divide their games into reusable components that have their own state, so a simple state modification does not invalidate game-wide assumptions made during a previous state. Induction based reasoning also becomes easier since the amount of code affected by a change of state is smaller since code is divided into packages. Furthermore, having reusable packages allow underlying constructs to be reused in other proofs that rely on similar primitives. It also allows for nice visualisations of package compositions as graphs.

It is worth noting that a proof of Yao's Garbled Circuit construction has already been expressed as a SSP by Brzuska and Oechsner [BO23]. This is a proof of the garbling scheme, rather than the Garbled Circuit protocol. This thesis intends to build on top of this work by converting Kolesnikov and Schneider's proof for the Free XOR garbling scheme under the LIN-RK-KDM assumption into a SSP. The choice is primarily driven by the fact that significant future work in garbling schemes such as Half-Gates [ZRE15] and Three-Halves [RR21] use it as a crucial component. The improvements in these schemes are not achievable without using Free XOR.

1.1.4 Bridging the concepts

Domino is a highly automated proof assistant for SSPs that allows expression of games, packages and oracles in a syntax intentionally very similar to pen-and-paper SSPs. Currently available as a standalone executable (‘crate’) in the Rust package manager (cargo), it includes a command-line interface that validates SSP structures, provides type-checking and provides a decent degree of error handling. It also comes with a minimal extension for the Visual Studio Code text editor that provides syntax highlighting. Domino uses an underlying Satisfiability Modulo Theory (SMT) solver to verify equivalence relations between games with different code, and also verifies reduction game-hops. It is worth noting that there is ongoing work in encoding and validating Brzuska and Oechsner’s state-separating proof for Yao’s garbling scheme in Domino [BERW26]. This work shows the feasibility of encoding the Free XOR garbling scheme in Domino, and therefore, we have this as an additional goal for this thesis. We chose to encode Applebaum’s variation of Free XOR in this thesis because of its closeness to Yao’s scheme - both of them use encryption to generate the garblings, instead of a hash-based one-time pad (OTP) used in Kolesnikov and Schneider’s work. While the hash-based construction is more efficient, it uses a stronger Random Oracle assumption, which is harder to translate to real life, whereas Applebaum’s scheme comes with a description of how one can construct a Linear, Related-Key, Key-Dependent-Message secure encryption scheme using an underlying Learning Parity with Noise (LPN) assumption.

It is worth mentioning that this thesis initially aimed to formally verify the Half-Gates garbling scheme, but ran into the research gap that exists between state-separating proofs for Yao’s garbling scheme, and Half-Gates. Since Half-Gates builds on top of the Free XOR scheme, this thesis marks the first steps towards formally verifying the Half-Gates scheme, going as far as Free XOR.

1.2 Objectives

The goals of this thesis can be summarized as follows:

1. To develop a State Separating pen-and-paper reduction proof for the Free XOR garbling scheme.

2. To encode this proof and verify its correctness in Domino.

1.3 Sustainability

While theoretical cryptography itself does not map well to any of the United Nations (UN) Sustainable Development Goal (SDG)s, since the indicators for each subgoal are very distinct, the following arguments can be made for the contributions that this thesis may make to the targets of various subgoals:

- **Target 9.5 - Enhance research and upgrade industrial technologies:** This work pushes the boundaries of research in verified proofs of cryptographic algorithms and protocols that are used to build software technologies.
- **Target 16.6 - Develop effective, accountable and transparent institutions:** This work is in the domain of privacy-preserving technologies, which when implemented into software can allow institutions to be more transparent while retaining the privacy of information.

1.4 Outline

The rest of this thesis is organized as follows: Chapter 2 contains background information on garbling schemes, optimizations, SSPs, and Domino. Chapter 3 details the converted pen-and-paper proof for Free XOR. Chapter 4 contains further details about the proof tool Domino, a brief overview on how to set it up, and an encoding of the state-separating proof in Domino. Chapter 5 discusses the advantages and challenges of working with Domino as opposed to plain pen-and-paper SSP. Chapter 6 contains concluding remarks and future work.

Chapter 2
Background

In this chapter, we introduce the important cryptographic building blocks needed in this thesis. Section 2.1 introduces Garbled Circuits, while Section 2.2 introduces garbling schemes, their security properties, and Yao's GS. Section 2.3 discusses the GS optimizations that are relevant to the work done in this thesis.

2.1 Garbled Circuits

A Garbled Circuit can be informally defined as a two-party protocol, in which both parties want to evaluate a shared function C together, without revealing their inputs to each other. The two parties participating in the protocol are called the Garbler (or the Garbled Circuit generator), and the Evaluator. The function C needs to be expressed as a boolean circuit if a Garbled Circuit protocol is used. Input hiding is achieved using something called *garbling*, which in practice is some form of encryption. Similarly, the final output is initially garbled, and needs to be ungarbled, which is then implemented using a corresponding decryption function.

A simple run of the protocol starts with both parties possessing their own input bit respectively (g, and e). The protocol proceeds as follows (ref. Figure 2.1):

1. The Garbler garbles the circuit C to obtain the Garbled Circuit C', along with some encoding information (enc) and decoding information (dec). These will be used to garbled and ungarble inputs and outputs.

2. The Garbler garbles its own input g into g'.

3. The Garbler then sends across C', g' and dec to the Evaluator.

4. The Evaluator still needs to garble its own input, but does not possess the encoding information, enc to do so. Therefore, both parties participate in what is known as a 1-out-of-2 *Oblivious Transfer* (OT) protocol [EGL82]. This

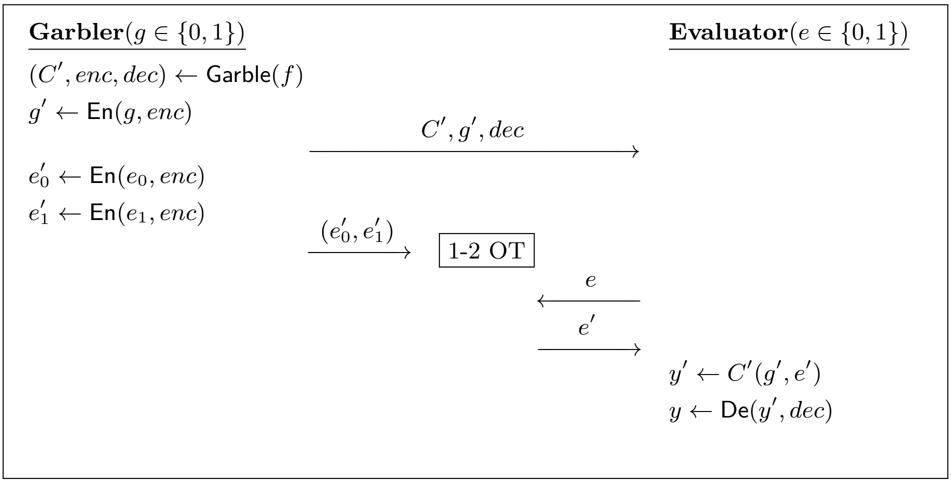


Figure 2.1: Garbled Circuit Protocol

protocol (adapted for GCs) requires the Garbler to garble both possible bit-values that the Evaluator may possess, and provide them to the OT protocol. Then, the Evaluator provides their chosen bit e to the OT protocol and the OT protocol returns the garbling of that bit (e'). The Garbler remains unaware of which garbling was sent to the Evaluator, and the Evaluator remains unaware of the value of the other garbling.

5. At this point, since the Evaluator possesses both garbled inputs (g' and e') and the Garbled Circuit (C'), it can compute the garbled output, y' by running the garbled inputs through the Garbled Circuit.

6. Finally, using the decoding information *dec*, the Evaluator can obtain y by ungarbling y'. This output is equivalent to the output of the function $C(g, e)$.

7. If necessary, the Evaluator can send the final output back to the Garbler.

The protocol can also be scaled up to support larger inputs from both parties. However, this would increase the complexity of the Oblivious Transfer protocol.

Both the Garbler and Evaluator are assumed to be *honest-but-curious*. This means that both parties adhere strictly to the protocol specification. However, they try to learn more than what is intended by using the information received from running the protocol, and from their own randomness and state. So, as an example, the Evaluator could try to obtain the Garbler's bit by ungarbling it, but will not succeed since the algorithm *dec* that the Evaluator receives does not ungarble the inputs.

This kind of adversary is equivalent to the 'Semi-Honest' adversary described by Lindell [Lin17].

2.2 Garbling Schemes

In the previous section, we discussed a 2PC Garbled Circuit protocol. However, it is easier to reason about a part of the security of such a protocol using a more generalized construct called a garbling scheme. A garbling scheme leaves out the Oblivious Transfer sub-protocol and focuses on the correctness and security properties of the garbling of the circuit and the input.

A garbling scheme [BHR12] can be expressed as a 5-tuple of algorithms

$$(\mathsf{Garble}(\mathsf{Gb}), \mathsf{Encode}(\mathsf{En}), \mathsf{Decode}(\mathsf{De}), \mathsf{GarbledEval}(\mathsf{Ev}), \mathsf{Eval}(\mathsf{ev}))$$

1. $\mathsf{Garble}(\mathsf{Gb})$ takes as input the function f and the security parameter 1^λ and returns (F, e, d), where F is the Garbled Circuit, e is the encoding information and d is the decoding information.
2. $\mathsf{Encode}(\mathsf{En})$ takes as input the initial input x and the encoding information e, and returns the garbled input X.
3. $\mathsf{GarbledEval}(\mathsf{Ev})$ takes as input the Garbled Circuit F and the garbled input X, and returns a garbled output Y.
4. $\mathsf{Decode}(\mathsf{De})$ takes input the garbled output Y and the decoding information d, and returns the ungarbled output y.
5. $\mathsf{Eval}(\mathsf{ev})$ takes as input the function f and the plain input x and returns $f(x)$. For the sake of simplicity, we denote $\mathsf{ev}(f, x)$ as $f(x)$, which is more familiar.

Relating this to the representation of the Garbled Circuit protocol in the previous section, the inputs g and e from the previous section are partitions of x. That is, x can be expressed as $x = g \| e$.

2.2.1 Properties

The correctness property of a garbling scheme describes its intended behavior. It says , $\forall f$, if $(F, e, d) \leftarrow\!\!\$\ \mathsf{Gb}(1^\lambda, f)$,

$$\forall x, \mathsf{De}(\mathsf{Ev}(F, \mathsf{En}(x, e)), d) = f(x)$$

Besides correctness, a garbling scheme can have one or more of the following security properties [BHR12]:

1. **Privacy**: Knowledge of (F, X, d) does not reveal anything beyond $f(x)$. This means that it is computationally infeasible for an Evaluator to obtain x, the initial input of the Garbler.

2. **Obliviousness**: Knowledge of (F, X) does not reveal anything beyond itself. This means that despite being able to solve the Garbled Circuit, the solver would not be efficiently able to map the garbled output to the real output without d.

3. **Authenticity**: Knowledge of (F, X) does not allow one to efficiently find a valid $Y' \neq \mathsf{Ev}(F, X)$.

Of these properties, *Privacy* is the one that is most important to us in this work, and it is what we will continue working with in the rest of the thesis.

2.2.2 Defining Privacy

Bellare, Hoang and Rogaway define privacy as two different cryptographic games: an *indistinguishability* based one (Game $\mathsf{PrvInd}_{G,\Phi}$), and a *simulation* based one (Game $\mathsf{PrvSim}_{G,\Phi,S}$) as described in Figure 2.2 [BHR12]. In these notations, and elsewhere, Φ refers to the side information function, which describes how much information about the circuit is public. In common usage, it is often either the whole circuit (denoted as $\Phi_{circ}(C)$), the topology of the circuit ($\Phi_{topo}(C)$), or the size of the circuit ($\Phi_{size}(C)$).

$G^b_{\mathsf{PrvInd}_{G,\Phi}}$

$\mathsf{GARBLE}(f_0, f_1, x_0, x_1)$

if $\Phi(f_0) \neq \Phi(f_1)$ **then**
  return $\perp$
if $f_0(x_0) \neq f_1(x_1)$ **then**
  return $\perp$
$(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f_b)$
$X \leftarrow \mathsf{En}(x_b, e)$
return (F, X, d)

$G^b_{\mathsf{PrvSim}_{G,\Phi,S}}$

$\mathsf{GARBLE}(f, x)$

if $b = 0$ **then**
  $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$
  $X \leftarrow \mathsf{En}(x, e)$
else
  $y \leftarrow \mathsf{ev}(f, x)$
  $(F, X, d) \leftarrow \mathcal{S}(1^\lambda, y, \Phi(f))$
return (F, X, d)

Figure 2.2: PrvInd and PrvSim Games - adapted from [BHR12]

In the indistinguishability based game (Game $\mathsf{PrvInd}^{\mathsf{b}}_{G,\Phi}$), the GARBLE oracle takes as parameters two pairs of functions and corresponding inputs, (f_0, x_0) and (f_1, x_1) such that $f_0(x_0) = f_1(x_1)$, and the bit b that defines which (f, x) pair GARBLE will use. A distinguishing adversary $\mathcal{D}$ is a polynomial-runtime program that tries to

guess the bit-value b that was used to generate the (F, e, d) tuple. An INITIALIZE procedure samples the value of b, and it is unknown to the adversary. GARBLE then uses the selected (f, x) to generate F, X and d and returns it to the adversary. The advantage of the distinguishing adversary here is defined as the difference in the probability that the adversary guesses the correct bit and the probability of guessing randomly $(P = 1/2)$.
If b' is $\mathcal{D}$'s guess,

$$\mathsf{Adv}_{\mathcal{D},G^0_{\mathsf{PrvInd}},G^1_{\mathsf{PrvInd}}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|$$

However, many follow-up works such as Half-Gates [ZRE15] and Free XOR [KS08; App16], use the simulation based paradigm (the PrvSim game from Figure 2.2). Since privacy here means that running the garbling and encoding algorithms Gb and En does not reveal anything about f or x, we let the simulator $\mathcal{S}$ access everything that a semi-honest party participating in the protocol would be able to see, i.e., the security parameter (1^λ), the output of the algorithm y, and the side information function of the circuit $\Phi(f)$. Then, this simulator tries to generate a $(\bar{F}, \bar{X}, \bar{d})$ tuple whose distribution is computationally indistinguishable from the one generated by actually running Gb and En. The advantage of an adversary $\mathcal{A}$ against the simulation game $G_{\mathsf{PrvSim}^{\mathsf{b}}_{G,\Phi,S}}$ would be defined as

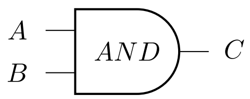
$$\mathsf{Adv}_{\mathcal{A},G^0_{\mathsf{PrvSim}},G^1_{\mathsf{PrvSim}}}(\lambda) = |\Pr\big[1 = \mathcal{A} \to G^0_{\mathsf{PrvSim}}\big] - \Pr\big[1 = \mathcal{A} \to G^1_{\mathsf{PrvSim}}\big]|$$

If the distributions of $(\bar{F}, \bar{X}, \bar{d})$ and (F, X, d) are indistinguishable to any adversary $\mathcal{A}$, the adversary cannot claim to have recovered anything more than what the simulator knew, and hence the privacy of f (excluding $\Phi(f)$) and x is preserved. For further reading on simulators, Lindell provides a comprehensive overview of simulators for beginners [Lin17].

2.2.3 Yao's Garbling Scheme

Before we get into the optimizations for garbling scheme, we must first define a base. In this work, we consider the garbling scheme model defined by Lindell and Pinkas [LP09], and will refer to it as Yao's garbling scheme. The garbling of a circuit f consists of n garbled gates, where n is the number of gates that compose the original circuit before garbling. Each (garbled and ungarbled) gate has two input wires (A and B) and one output wire (C). Each of these ungarbled wires can have two possible values on each wire - 0 or 1 (corresponding to real-life boolean circuits). We will use a, b, and c to refer to the actual bit-values of each wire. Figure 2.3(a) shows this as a diagram, and the truth table of an AND gate.

Now, we map each of the possible values on each wire to a random string of length λ denoted by the wire name and the bit-value it was mapped to. This random



(a) AND gate

a	b	$a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

Input labels	Ciphertext
(A_0, B_0)	X_1
(A_0, B_1)	X_2
(A_1, B_0)	X_3
(A_1, B_1)	X_4

(c) Generating a garbled table

Figure 2.3: An AND gate and its garbled form.

value is called the wire label for that particular wire and bit-value combination. For example, the random value mapped to the 0-input on wire A would be called A_0. After repeating this process to get all six wire labels $(A_0, A_1, B_0, B_1, C_0, C_1)$, we can generate the garbled table. It is worth noting that this mapping from the wire's bit-value to the wire label corresponds to the encoding information e used in the En function in the description of a garbling scheme in Section 2.2.

The garbled table contains 4 rows, like the truth table, and it contains encryptions of the wire label of the output value of each row ($C_{a \wedge b}$), under two keys - the wire labels of the corresponding inputs (A_a,B_b) that generated the output $a \wedge b$ in that row of the truth table. For example, if we consider row 1 of Figure 2.3(c), we would have

$$X_1 = \mathsf{Enc}(A_0, \mathsf{Enc}(B_0, C_0)).$$

These rows are then permuted so that the relation between the wire labels and the actual bit-values remains hidden.

We know from the protocol description in Section 2.1, that besides the garbling of the circuit, the Garbler also sends the 'garbling' of the inputs to the Evaluator. These garblings of the input refers to the wire labels corresponding only to the inputs that the parties want to evaluate using the circuit. Thus, the Evaluator receives the permuted garbled table, as well as the wire labels of the inputs (which, for a single wire, can either be 0 or 1, but not both simultaneously). We observe from Figure 2.4 that decrypting any row requires possession of a specific A wire label and a specific B wire label. This means that if the Evaluator receives only one A label and one B label, they cannot decrypt any row other than the one that was encrypted using those labels since at least one of the two decryptions will fail. This preserves the privacy of the original bit-value of wire C. However, this requires that the Garbler provides a mapping from the final output wire labels to the real bit-value of that wire. This would correspond to the decoding information d used in the function De.

In this garbling scheme, we assume that when trying to decrypt a ciphertext without the right keys, the decryption will fail and the decryptor will know that it has failed. This can be achieved by concatenating some specific, publicly-known padding in the wire labels for wire C. This is related to a property called special correctness that where attempting to decrypt a ciphertext without a valid key will result in a decryption failure message with overwhelming probability. The character $\perp$ is commonly used to show such a result in cryptographic notation. Special correctness is a property used later in our work as well.

Ciphertext	Value
X_1	$\mathsf{Enc}(A_0, \mathsf{Enc}(B_0, C_0))$
X_2	$\mathsf{Enc}(A_0, \mathsf{Enc}(B_1, C_0))$
X_3	$\mathsf{Enc}(A_1, \mathsf{Enc}(B_0, C_0))$
X_4	$\mathsf{Enc}(A_1, \mathsf{Enc}(B_1, C_1))$

Figure 2.4: Garbled table for an AND gate

For a larger circuit, garbled tables are generated for each garbled gate in the original circuit, and all of them are transmitted to the Evaluator. In addition to this, a larger circuit would have a larger input as well, so the wire labels corresponding to every input bit is also sent.

2.3 Optimizations to Yao's Garbling Scheme

In the last two decades, there have been many optimizations that improve the efficiency of Yao's garbling scheme, in terms of the computational costs of garbling, evaluating, and also in terms of the bandwidth consumption when transmitting the Garbled Circuit. In this section, we discuss the optimizations that are relevant to our work, namely - Point and Permute (P&P) [BMR90], Garbled Row Reduction (GRR) [NPS99], Free XOR [KS08], and Half-Gates [ZRE15].

2.3.1 Point and Permute

On observing Yao's garbling scheme, it quickly becomes obvious that the Evaluator may have to decrypt 4 rows of the garbled table in order to receive the correct output wire label. Yao's garbling scheme also requires special correctness, which is not difficult to implement, but there is a way to avoid the whole problem in the first place. This technique, now called Point and Permute, is based on the value

λ_{W-1} defined in the garbled circuit protocol of Beaver et al. [BMR90]. Note that λ continues being as the security parameter in this thesis.

To put it simply, for each wire W in the circuit, with wire labels W_0 and W_1, each of length λ, an extra bit is sampled (called the permute or color bit). Let us call this bit l_W. Then, for each wire W,

$$W'_0 = W_0 \| (l_W), \text{ and}$$

$$W'_1 = W_1 \| (l_W \oplus 1)$$

Now, when generating a garbled table, the ciphertexts are ordered based on the permute bit of the wire labels used to encrypt it. For example, if we say that the encryption generated by wire labels A'_a and B'_b would be in row $2l_a + l_b + 1$, and if A'_0 and B'_0 both have the permute bit 1, the garbled table would be ordered as in Figure 2.5. Then, the Evaluator can use the last bit (the permute bit) of both input wire

l_a	corresp. A'_a	l_b	corresp. B'_b	$C'_c = C'_{a \wedge b}$	Ciphertext
0	A'_1	0	B'_1	C'_1	$\mathsf{Enc}(A'_1, \mathsf{Enc}(B'_1, C'_1))$
0	A'_1	1	B'_0	C'_0	$\mathsf{Enc}(A'_1, \mathsf{Enc}(B'_0, C'_0))$
1	A'_0	0	B'_1	C'_0	$\mathsf{Enc}(A'_0, \mathsf{Enc}(B'_1, C'_0))$
1	A'_0	1	B'_0	C'_0	$\mathsf{Enc}(A'_0, \mathsf{Enc}(B'_0, C'_0))$

Figure 2.5: Garbled table for an AND gate with Point and Permute

labels to identify which row to decrypt in order to obtain the output wire label. This process continues until the end of the circuit is processed since each output wire label contains a permute bit as well, and all the garbled tables are ordered based on these permute bits. However, this requires that the Evaluator and Garbler use the same method of permuting the rows, which can be defined in the protocol specification.

This method reduces the number of ciphertexts to be decrypted by the Evaluator to 1, which makes the evaluation faster. Point and Permute also removes the need for the special correctness property that Yao's Garbled Circuit had. It is also of minor note that the length of each wire label increases by one bit, but the security parameter λ remains the same.

2.3.2 Garbled Row Reduction-3

Garbled Row Reduction, also known as Garbled Row Reduction-3 refers to a scheme first introduced by Naor et al. [NPS99, Section 2.5] under the heading 'Optimizing the circuit size'. As discussed briefly in Chapter 1, this optimization allows the number of rows in the garbled table of a boolean gate with two inputs to be reduced to *three* instead of *four*, thus reducing the bandwidth overhead by 25%. The 3 in the name of the scheme refers to the three rows in the garbled table.

This technique works by setting one row of the garbled table of each gate to 0, and therefore assuming it implicitly, instead of sending it. Let us say that row $X_{a,b}$ of the garbled table of a gate is the one being omitted. Here, a and b are the bit-values of the input wires A and B for that gate. Since we would normally compute that row as

$$X_{a,b} = \mathsf{Enc}(A_a, \mathsf{Enc}(B_b, C_c)), \text{ and}$$

$$\text{since } X_{a,b} = 0^\lambda,$$

it would mean that the Evaluator could compute

$$C_c = \mathsf{Enc}^{-1}(B_b, \mathsf{Enc}^{-1}(A_a, 0)).$$

Initially, it might seem that this is insecure since the wire label C_c depends on A_a and B_b instead of being randomly sampled, it is not actually the case as the Evaluator cannot see the value for C_c unless it has A_a and B_b, and if it has those two wire labels, it cannot have the wire labels needed to view the $C_{c\oplus 1}$ present in the other rows of the garbled table.

This technique can also be combined with the Point and Permute (P&P) optimization described in Section 2.3.1, where the deleted row would correspond to the permute bits $(0, 0)$. However, in this work, we do not deal with the Point and Permute optimization to reduce the complexity of the proof, especially in Domino.

2.3.3 Free XOR

The Free XOR optimization was a big step in the optimization of Garbled Circuits, allowing the transmission of an XOR gate for free, i.e., without the need to generate, send, or evaluate a garbled table for an XOR gate. This is a big improvement when we consider that circuits can be optimized to maximize the number of XOR gates. This technique was introduced by Kolesnikov and Schneider [KS08].

Instead of sampling each of the wire labels for every wire independently, the Free XOR scheme samples a global offset $\Delta \in \{0,1\}^\lambda$ such that for each wire W,

$$W_1 = W_0 \oplus \Delta, \text{ where } W_0 \leftarrow\$ \{0,1\}^\lambda$$

Then, for each XOR gate with input wires A,B and output wire C, the Evaluator can compute the value of the active wire label of C as $C = A \oplus B$ (see Figure

A_a	B_b	$C_c = A_a \oplus B_b$
A_0	B_0	$A_0 \oplus B_0$
A_0	$B_0 \oplus \Delta$	$A_0 \oplus B_0 \oplus \Delta$
$A_0 \oplus \Delta$	B_0	$A_0 \oplus B_0 \oplus \Delta$
$A_0 \oplus \Delta$	$B_0 \oplus \Delta$	$A_0 \oplus B_0$

Figure 2.6: Evaluation of an XOR gate in Free XOR

2.6). Since this calculation of the active wire label of C is always true for XOR gates, the Garbler can just indicate the presence of an XOR gate, omitting the cost of encrypting the output wire labels. These garblings also do not reveal the wire bit-values they correspond to, so privacy remains intact. However, this method of garbling gates requires the hash or encryption function used to garble the output wire labels must satisfy a property called 'Correlation Robustness', since the wire labels used as keys and the output wire label are all in relation to the circuit's global offset Δ.

The notation used in Kolesnikov and Schneider's Free XOR paper is different from the usual encryption under two keys we have seen so far [KS08]. Their scheme uses a hash-based notation to generate a one-time pad which is then XOR'ed with the output wire label. As mentioned in Chapter 1, this scheme was shown to be secure under the 'Random Oracle'(RO) Model [BR93], with the authors also claiming security under some version of correlation-robustness of the hash function (the authors do not describe the exact kind of assumption), which is a weaker assumption than the RO model. They defined the ciphertext correspoding to input bit-values a and b in the garbled table as follows: Given a Random Oracle $H : \{0,1\}^* \mapsto \{0,1\}^{\lambda+1}$, and a gate at index i with operation $op \in \{AND, OR, ...\}$,

$$X_{a,b} = H(A_a \| B_b \| i) \oplus C_c$$

Choi et al. then went on to show that general correlation-robustness was insufficient, and described the exact form of the assumption needed and called it circular correlation-robustness [CKKZ12]. This new assumption is still weaker than the RO model, but stronger than correlation-robustness. Later, Applebaum also proved the security of the Free XOR scheme under a different assumption called 'Linear, Related-Key, Key-Dependent-Message' (LIN-RK-KDM) security for encryption schemes, which uses the following definition for encryption of each row.

$$R_{a,b} \leftarrow\$ \{0,1\}^{\lambda}$$

$$X_{a,b} = (\mathsf{Enc}(A_a, R_{a,b}), \mathsf{Enc}(B_b, R_{a,b} \oplus C_{op(a,b)}))$$

In Applebaum's scheme, the Evaluator decrypts both ciphertexts using the input wire labels it receives, and XORs them together to obtain the active output label C_c. We observe that there are two encryption operations per garbled table row, namely $\mathsf{Enc}(A_a, R_{a,b})$ and $\mathsf{Enc}(B_b, R_{a,b} \oplus C_{op(a,b)})$ (eight in total per gate), compared to Kolesnikov and Schneider's scheme, which requires one hash operation $H(A_a \| B_b \| i) \oplus C_c$ per row. Despite this reduction in efficiency, Applebaum's scheme is interesting because it was published with a detailed method on how to build a LIN-RK-KDM secure encryption scheme out of the Learning Parity with Noise (LPN) assumption. This, in conjunction with the fact that this construction is based on encryption like in Yao's garbling scheme, is what made it attractive for us to use Applebaum's scheme as the Free XOR proof of choice in this thesis. A detailed description of the LPN assumption is available in their paper[App16].

It is also very important to mention that Kolesnikov and Schneider noted that NOT gates can be implemented in a garbled circuit for free by simply flipping the bit-value of the wire they serve as input to [KS08, Section 3.1].

2.3.4 Half-Gates

Half-Gates was a novel technique introduced by Zahur et al. [ZRE15] which uses the splitting of an AND gate into two half-AND gates and XORing the results to reduce the number of rows to be transmitted. While splitting the gates may seem counterintuitive, the inputs to one of the half-gates are known by the Garbler and the inputs to the other half-gate are known by the Evaluator, and therefore, the outputs of those gates turn into a function of these known values, reducing each AND half-gate to two rows each. Then, using the GRR optimization, each of these two row half-gates are reduced to one row each. The XOR operation itself is free since this scheme builds on top of Free XOR, and we arrive at 2 total ciphertexts per AND gate. The detailed treatment of this scheme is beyond our scope; see [ZRE15].

Chapter 3
Pen-and-Paper State-Separating Proof

This chapter introduces the notion of a State Separating Proof (SSP), and how it differs from a traditional code-based game-playing proof. Then, we define our pen-and-paper state-separating proof for the Free XOR garbling scheme here.

3.1 State Separating Proofs

State separating proofs are a style of proofs built upon code-based game-playing proofs discussed by Bellare and Rogaway [BR06]. They define code-based game-playing as proofs that encompass an adversary's interaction with its environment using code (could be pseudocode or any other formalized programming language), and these interactions are referred to as a game. The game contains subroutines that initialize the state of the game, then run the adversary program. The adversary can call oracles, which are functions exposed by the game, and the game concludes by running a finalize subroutine which outputs a bit from the adversary. The adversary has a similar interaction between two different versions of the game, one in a real world and one in an ideal world, whose behaviors are parameterized by a bit $b = 0$ and $b = 1$, respectively. The success of the adversary is characterized by its 'advantage', defined as the difference in the probability of the adversary returning the same bit in each game. In cryptographic notation, given two games G_{real} and G_{ideal},

$$\mathsf{Adv}_{\mathcal{A},G^0_{real},G^1_{ideal}}(\lambda) = |\Pr[A \to G_{real} = 1] - \Pr[A \to G_{ideal} = 1]|$$

Games and Oracles are constructs of a game-playing proof.

- **Oracles** are defined as a piece of code that provides some functionality. They usually either return a value, or modify the state of the Game.
- **Games** are a complete set of oracles that do not call other oracles, an initialize procedure and a finalize procedure that returns a bit. An adversary interacts with a game.

However, state-separating proofs differ by introducing the notion of a package. A package has its own local state, and it functions as an intermediate layer between a game and a package. Games contain packages, and packages contain oracles. In a state-separating proof [BDF+18], they are defined as follows:

- **Oracles** are a piece of executable code. They usually either return a value or modify the state of the package they are contained in. Oracles in a package cannot call each other to prevent recursion.
- **Packages** are defined as a collection of related oracles with a shared state.
- **Games** are defined as a package with no dependencies on oracles of other packages ($[G \rightarrow] = \varnothing$).
- An **adversary** can be defined as a special package that is not a dependency of another package. It always outputs a single bit on termination ($[\rightarrow A] = \varnothing$).

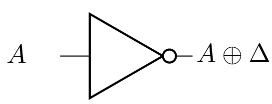
State separating proofs allow us to restrict the access to the state of each package to oracles in that package, and only provide the oracles themselves as an interface to other packages. This allows them to be highly modular and composable, allowing us to reuse a package used in one reduction in another, all while allowing changes in state that do not make changes to the whole game. If these changes cascaded to the whole game, it would require the cryptographer to once again verify the correctness of the game in the new state. [BDF+18] formally defines the constructs we use here, and the mathematical properties of packages such as associativity and compositions. We slightly diverge from their notation of sequential composition by using the $\rightarrow$ operator instead of $\circ$.

3.2 On the Privacy of the Garbling of Various Gates

In our work, we are concerned with the garbling of NOT, XOR and AND gates, since every boolean circuit can be represented with that combination of gates (functional completeness of boolean gates). Despite the fact that NOT and AND gates by themselves are functionally complete, we are interested in XOR gates since they are the basis on which the Free XOR optimization improves the efficiency of evaluating a Garbled Circuit. We discuss these gates in order:

3.2.1 NOT Gate

Bit-value	Wire label
0	W
1	$W \oplus \Delta$



Bit-value	Wire label
0	$W \oplus \Delta$
1	W

Figure 3.1: Wire label mappings before (left) and after (right) a NOT gate.

Kolesnikov and Schneider note that NOT gates do not need to be garbled separately at all [KS08, Section 3.1]. A NOT gate simply inverts the mapping between the bit-value of a wire and its wire label, i.e., if the wire label W_b corresponds to bit b before encountering a NOT gate, the wire label W_b will correspond to bit $b \oplus 1$ after the NOT gate (Figure 3.1 describes this inversion). Since NOT gates are not garbled, they are not part of the security proof of the privacy of the garbling of a circuit.

3.2.2 XOR Gate

Similarly to NOT gates, XOR gates are garbled for 'free' in the Free XOR scheme. What this means in practice is that the Evaluator directly computes the output wire label of an XOR gate using the input wire labels (as discussed in Section 2.3.3). There is no garbled table that the Evaluator receives, and no decryption that takes place, and the security of the XOR gate depends directly on the secrecy of the circuit offset Δ. As we will see in Section 3.2.3, AND gates generate their garbled tables using encryption schemes that are LIN-RK-KDM secure. This means that the relation caused between wire labels by deriving the output wire label of an XOR gate by XORing the input wire labels that may both be functions of the offset Δ is accounted for in the underlying assumption we use to prove the security of an AND gate. That is to say, that even in this related-key and key-dependent message setting, there is no privacy loss from an XOR gate in the Free XOR scheme, if Δ remains secret.

3.2.3 AND Gate

The privacy of the garbling of AND gates in the Free XOR scheme becomes the main focus of this work. As mentioned in Section 2.3.3, we base our state-separating proof on Applebaum's proof in [App16].

Since we use the simulation based definition of privacy (discussed in Section 2.2.2), we define the garbling of an AND gate in the Free XOR scheme as the security game $G^0_{\mathsf{Sim\text{-}FreeXOR}}$ (defined in Figure 3.2). The game exposes a GARBLE oracle that provides the core functionality of garbling an AND gate. It first obtains the evaluation of the circuit f with the input x to determine the output it has to garble. The input x is partitioned into the bit-values for wire A and B. Then, it samples a random offset value, and wire labels for the two active input wires and active output wire. It also samples four random values, one for each row of the garbled table. The order in which these values are used is irrelevant since they are not used across multiple rows of the garbled table, therefore, they can be used independently of the index of the garbled table, as long as they are not reused. The garbled table is then generated as described in Section 2.3.3. The garbled inputs are the wire labels sampled corresponding to the active wire bit-values. The decoding information d is

$G^0_{\textsf{Sim-FreeXOR}}$

$\underline{\underline{\textsf{GARBLE}(f, x)}}$

$c \leftarrow f(x)$
$(a, b) \leftarrow x$
$\Delta \leftarrow\!\!\$ \{0,1\}^\lambda$
$A_a, B_b, C_c \leftarrow\!\!\$ \{0,1\}^\lambda$
$A_{a\oplus 1} \leftarrow A_a \oplus \Delta; B_{b\oplus 1} \leftarrow B_b \oplus \Delta; C_{c\oplus 1} \leftarrow C_c \oplus \Delta$
$R_{00}, R_{01}, R_{10}, R_{11} \leftarrow\!\!\$ \{0,1\}^\lambda$
F : Table
$F[a][b] \leftarrow\!\!\$ (\textsf{Enc}(A_a, R_{00}), \textsf{Enc}(B_b, R_{00} \oplus C_{a\wedge b}))$
$F[a][b\oplus 1] \leftarrow\!\!\$ (\textsf{Enc}(A_a, R_{01}), \textsf{Enc}(B_{b\oplus 1}, R_{01} \oplus C_{a\wedge(b\oplus 1)}))$
$F[a\oplus 1][b] \leftarrow\!\!\$ (\textsf{Enc}(A_{a\oplus 1}, R_{10}), \textsf{Enc}(B_b, R_{10} \oplus C_{(a\oplus 1)\wedge b}))$
$F[a\oplus 1][b\oplus 1] \leftarrow\!\!\$ (\textsf{Enc}(A_{a\oplus 1}, R_{11}), \textsf{Enc}(B_{b\oplus 1}, R_{11} \oplus C_{(a\oplus 1)\wedge(b\oplus 1)}))$
$X \leftarrow (A_a, B_b)$
$d \leftarrow \{C_c : c\}$
return (F, X, d)

Figure 3.2: Code for $G^0_{\textsf{Sim-FreeXOR}}$

set as a map from the active wire label for the output wire to the bit-value of the output wire.

$G^b_{\textsf{LIN-RK-KDM}}$

State:
Δ : global circuit offset

$\underline{\underline{\textsf{ENC}(key, msg, z \in \{0,1\})}}$

if $\Delta = \bot$: // If Δ is unset
  $\Delta \leftarrow\!\!\$ \{0,1\}^\lambda$ // Sample a fresh Δ
if $b = 0$:
  $ct \leftarrow\!\!\$ \textsf{Enc}(key \oplus \Delta,\ msg \oplus z \cdot \Delta)$
if $b = 1$:
  $ct \leftarrow\!\!\$ \textsf{Enc}(key \oplus \Delta,\ 0^{|msg|})$
return ct

Figure 3.3: Code for $G^b_{\textsf{LIN-RK-KDM}}$

Applebaum defines the notion of LIN-RK-KDM security of a symmetric encryption scheme $(\textsf{Enc}, \textsf{Dec})$ as the computational indistinguishability of the two games $\textsf{Real}_s$

and Fake_s [App16, Section 3.2], where

$$\mathsf{Real}_s : (\Delta, M, b) \mapsto \mathsf{Enc}_{S+\Delta}(M + bS), \text{ and,}$$

$$\mathsf{Fake}_s : (\Delta, M, b) \mapsto \mathsf{Enc}_{S+\Delta}(0^{l \times N}).$$

We redefine the same security notion to better fit our SSP setting:

Definition 3.1. An encryption scheme is said to be LIN-RK-KDM secure if any adversary $\mathcal{A}$ interacting with $G^b_{\mathsf{LIN-RK-KDM}}$ (Figure 3.3) in the real and ideal world cannot distinguish between them with more than negligible advantage, i.e.,

$$\mathsf{Adv}(\mathcal{A}, G^0_{\mathsf{LIN-RK-KDM}}, G^1_{\mathsf{LIN-RK-KDM}}) \leq \mathsf{negl}(\lambda).$$

Our $G^b_{\mathsf{LIN-RK-KDM}}$ provides an interface ENC that takes the key, message and an extra parameter z as input, and returns an encryption of the message in the real game, and an encryption of zeroes in the ideal game (Figure 3.3). It is important to note that the real LIN-RK-KDM game does not simply return an encryption under the key used as input to the ENC oracle. It XORs this key by an offset of Δ, and optionally XORs the same offset to the message, depending on the value of z.

Since we now have a base assumption to build from, we now try to reduce the security of the real and simulated garbling games (Figures 3.2 and 3.4) to the LIN-RK-KDM assumption.

Theorem 3.2. *If* Enc *is the LIN-RK-KDM secure encryption scheme used in the Free XOR garbling scheme, then for f={AND} and* $\forall x \in \{0,1\}^2$*, there exists a probabilistic, polynomial-time (PPT) simulator* $G^1_{\mathsf{Sim\text{-}FreeXOR}}$ *and reduction* $\mathcal{R}$ *such that for all PPT adversaries* $\mathcal{A}$*,*

$$\mathsf{Adv}(\mathcal{A}, G^0_{\mathsf{Sim\text{-}FreeXOR}}, G^1_{\mathsf{Sim\text{-}FreeXOR}}) \leq \mathsf{Adv}(\mathcal{A} \rightarrow \mathcal{R}, G^0_{\mathsf{LIN-RK-KDM}}, G^1_{\mathsf{LIN-RK-KDM}})$$

We now describe the simulation strategy used by $G^1_{\mathsf{Sim\text{-}FreeXOR}}$ (Figure 3.4). The simulator receives as input the output of the circuit evaluation, $f(x)$, which corresponds to the bit-value of the output wire of the AND gate. It then samples random bitstrings for the wire labels corresponding to the zero bit-values on the input and output wires. It also samples the global circuit offset Δ, and random values for each row of the garbled table. The correctness property of garbling requires that when the Evaluator evaluates a garbled table, they are able to decrypt exactly one row of the table with the input wire labels they have. It also requires that the final decrypted output wire label corresponds to the correct function evaluation $f(x)$.

Since the simulator knows $f(x)$, the simulation strategy involves mapping one of the output wire labels (our simulator in Figure 3.4 chooses C_0) to the bit-value $f(x)$, and

$G^1_{\textsf{Sim-FreeXOR}}$

$\underline{\underline{\textsf{GARBLE}(f(x),\ \Phi(f))}}$

$c \leftarrow f(x)$

$\Delta \leftarrow\!\!\$\ \{0,1\}^\lambda$ // global offset

$A_0, B_0, C_0 \leftarrow\!\!\$\ \{0,1\}^\lambda$

$A_1 \leftarrow A_0 \oplus \Delta$

$B_1 \leftarrow B_0 \oplus \Delta$

$C_1 \leftarrow C_0 \oplus \Delta$

$R_{00}, R_{01}, R_{10}, R_{11} \leftarrow\!\!\$\ \{0,1\}^\lambda$

F : Table

$F[0][0] \leftarrow\!\!\$\ (\textsf{Enc}(A_0, R_{00}), \textsf{Enc}(B_0, R_{00} \oplus C_0))$

$F[0][1] \leftarrow\!\!\$\ (\textsf{Enc}(A_0, R_{01}), \textsf{Enc}(B_1, 0^\lambda))$

$F[1][0] \leftarrow\!\!\$\ (\textsf{Enc}(A_1, 0^\lambda), \textsf{Enc}(B_0, R_{10} \oplus C_1))$

$F[1][1] \leftarrow\!\!\$\ (\textsf{Enc}(A_1, 0^\lambda), \textsf{Enc}(B_1, 0^\lambda))$

$X \leftarrow (A_0, B_0)$

$d \leftarrow \{C_0 : c\}$

return (F, X, d)

Figure 3.4: Code for $G^1_{\textsf{Sim-FreeXOR}}$

making sure that row that the adversary can decrypt actually encrypts the chosen output wire label. We also need to ensure that if the adversary receives wire labels A_0 and B_0 as the garbled input X, then every encryption on the table that uses these as the key is encrypted with real messages, but for the other encryptions, we can forge them by encrypting zeroes. This concludes the generation of the garbled table F. The garbled input X is a tuple of the active wire labels of the first and second wire, which we assumed to be A_0 and B_0 in our simulator. The choice only matters to select which row of the garbled table is available to the adversary for decryption. Finally, the decoding information d is a map from our chosen active output wire label that was encrypted with the active input wire labels, to the real value of $f(x)$ that is available to the adversary. Finally, the adversary returns this (F, X, d) tuple.

Another problem we quickly realise is that the two GARBLE oracles expect different inputs. GARBLE in the real game takes a circuit f, which in this case is a simple AND operation, and it also takes the inputs to the circuit, x. The GARBLE oracle in the ideal game however, takes $f(x)$ and $\Phi(f)$. To solve this incompatibility, we provide another game $G^b_{wrapper}$ that provides a shared interface GBL that can be composed with the real and ideal $G^b_{\textsf{Sim-FreeXOR}}$ games. However, we do not include

$G^0_{\mathsf{Wrapper},Gbl}$	$G^1_{\mathsf{Wrapper},Sim}$
$\underline{\underline{\mathsf{GBL}(f,x)}}$	$\underline{\underline{\mathsf{GBL}(f,x)}}$
$(F,X,d) \leftarrow\$ \mathsf{GARBLE}(f,x)$	$(F,X,d) \leftarrow\$ \mathsf{GARBLE}(f(x),\Phi(f))$
return (F,X,d)	**return** (F,X,d)

Figure 3.5: Code of $G^b_{wrapper}$

this in our proof in order to keep it simple to understand and translate into Domino code.

Lemma 3.3. (Real code equivalence). *Let $\mathcal{R}$ be the reduction defined in Figure 3.6. Then,*

$$\mathcal{R} \to G^0_{\mathsf{LIN-RK-KDM}} \overset{code}{\equiv} G^0_{\mathsf{Sim\text{-}FreeXOR}}.$$

$\mathcal{R}$

$\underline{\underline{\mathsf{GARBLE}(f,x)}}$

$c \leftarrow f(x)$
$(a,b) \leftarrow x$
$A_a, B_b, C_c \leftarrow\$ \{0,1\}^\lambda$
$R_{00}, R_{01}, R_{10}, R_{11} \leftarrow\$ \{0,1\}^\lambda$
F : Table
$F[a][b] \leftarrow\$ (\mathsf{Enc}(A_a, R_{00}), \mathsf{Enc}(B_b, R_{00} \oplus C_c))$
$F[a][b\oplus 1] \leftarrow\$ (\mathsf{Enc}(A_a, R_{01}), \mathsf{ENC}(B_b, R_{01} \oplus C_c, a \wedge (b \oplus 1)))$
$F[a\oplus 1][b] \leftarrow\$ (\mathsf{ENC}(A_a, R_{10} \oplus C_c, (a\oplus 1) \wedge (b)), \mathsf{Enc}(B_b, R_{10}))$
$F[a\oplus 1][b\oplus 1] \leftarrow\$ (\mathsf{ENC}(A_a, R_{11}, (a\oplus 1) \wedge (b \oplus 1)),$
$\qquad \mathsf{ENC}(B_b, R_{11} \oplus C_c, (a\oplus 1) \wedge (b \oplus 1)))$
$X \leftarrow (A_a, B_b)$
$d \leftarrow \{C_c : c\}$
return (F,X,d)

Figure 3.6: Code for Reduction $\mathcal{R}$

We describe the reduction $\mathcal{R}$ which can be composed with $G^b_{\mathsf{LIN-RK-KDM}}$. The interface to the reduction is a GARBLE oracle. It initially computes $f(x)$ and samples three bitstrings A_a, B_b and C_c of length λ assuming a,b and c are the active wire labels. It also samples four random bitstrings of length λ, just like the real $G^0_{\mathsf{Sim\text{-}FreeXOR}}$. However, unlike that game, the reduction does not sample its own Δ. This is because Δ is part of the state of the LIN-RK-KDM game, and therefore, the reduction must rely on calling the ENC oracle of the LIN-RK-KDM game in order to generate any of the ciphertexts that require inactive wire labels. The rest of the reduction follows the same steps as $G^0_{\mathsf{Sim\text{-}FreeXOR}}$, and inlining the ENC oracle gives us code that is equivalent ($\overset{code}{\equiv}$) to $G^0_{\mathsf{Sim\text{-}FreeXOR}}$, except for one important difference. For the calculation of $F[a \oplus 1][b]$, $\mathcal{R}$ encrypts the output wire label in the first ciphertext, as opposed to the second one which is where $G^0_{\mathsf{Sim\text{-}FreeXOR}}$ encrypts it. This is done because only the first encryption calls the ENC oracle, which can add an offset of Δ to the message. The correctness of the scheme still remains intact since XORing the messages still returns the correct value of the output wire label.

Thus, Lemma 3.4 provides a sufficient condition for the the code equivalence established in Lemma 3.3 to hold. Specifically,

Lemma 3.4. *Let* Enc *be a LIN-RK-KDM secure encryption scheme. Then,*

$$\begin{array}{lll} & (\mathsf{Enc}(A_{a\oplus 1}, R_{10} \oplus C_{c\oplus((a\oplus 1)\wedge b)}), & \mathsf{Enc}(B_b, R_{10})) \\ \overset{c}{\approx} & (\mathsf{Enc}(A_{a\oplus 1}, R_{10}), & \mathsf{Enc}(B_b, R_{10} \oplus C_{(a\oplus 1)\wedge b})) \end{array}$$

$$\implies \mathcal{R} \to G^0_{\mathsf{LIN-RK-KDM}} \overset{code}{\equiv} G^0_{\mathsf{Sim\text{-}FreeXOR}}$$

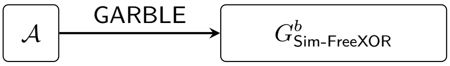
Now, we move on to showing code equivalence between the ideal games.

Lemma 3.5. (Ideal code equivalence). *Let $\mathcal{R}$ be the reduction defined in Figure 3.6. Then,*

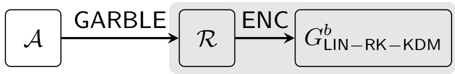
$$\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}} \overset{code}{\equiv} G^1_{\mathsf{Sim\text{-}FreeXOR}}.$$

On inlining oracle ENC of $G^1_{\mathsf{LIN-RK-KDM}}$ into $\mathcal{R}$, we obtain code that is very similar to $G^1_{\mathsf{Sim\text{-}FreeXOR}}$. This is not immediately obvious, so we define the code of $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$ in Appendix A.1. $G^1_{\mathsf{Sim\text{-}FreeXOR}}$ also requires some small transformations to show code equivalence, and therefore we define Game $G^{1\prime}_{\mathsf{Sim\text{-}FreeXOR}}$ in Appendix A.2. Here, we would like to discuss the transformations we have made. The composition of $\mathcal{R}$ to oracle ENC is quite straightforward, we inherit a state from $G^1_{\mathsf{LIN-RK-KDM}}$ which stores the global offset Δ. We also add code to $\mathcal{R}$ to sample Δ, however, this can be inlined without the check for an already existing Δ, since ENC is not called from outside of this game, and therefore we will never need to redefine it.

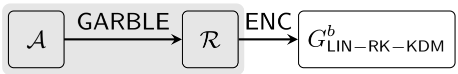
When rearranging $G^1_{\mathsf{Sim\text{-}FreeXOR}}$ into $G^{1\prime}_{\mathsf{Sim\text{-}FreeXOR}}$, we define constant variables a and b which are set to 0, simply to match the code in $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$. We also modify all the wire labels so that we refer to their bit-values using a, b and c instead of zeroes and ones. After making these changes, we can assert the following:



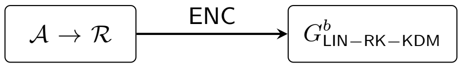
(a) Adversary $\mathcal{A}$ calls oracle GARBLE of Sim-FreeXOR Game



(b) Sim-FreeXOR game $\overset{code}{\equiv}$ $\mathcal{R} \to$ LIN-RK-KDM game



(c) Associativity allows us to compose $\mathcal{A} \to \mathcal{R}$



(d) The composed adversary $\mathcal{A} \to \mathcal{R}$ calls oracle ENC of the LIN-RK-KDM game

Figure 3.7: A visualization of the adversary transformation in the proof in Section 3.2.3

Lemma 3.6. *Let* Enc *be a LIN-RK-KDM secure encryption scheme. Then,*

$$\begin{aligned} & (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \quad \mathsf{Enc}(B_b, R_{10})) \\ \overset{c}{\approx} \; & (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \quad \mathsf{Enc}(B_b, R_{10} \oplus C_c \oplus \Delta)) \end{aligned}$$

$$\implies \mathcal{R} \to G^1_{\text{LIN-RK-KDM}} \overset{code}{\equiv} G^1_{\text{Sim-FreeXOR}}$$

Therefore, under the following assumptions 3.1 and 3.2 (which are the respective hypotheses of lemmata 3.4 and 3.6), lemmata 3.3 and 3.5 follow:

Assumption 3.1.

$$\mathsf{Enc}(A_{a\oplus 1}, R_{10} \oplus C_{c\oplus((a\oplus 1)\wedge b)}), \mathsf{Enc}(B_b, R_{10}) \overset{c}{\approx} \mathsf{Enc}(A_{a\oplus 1}, R_{10}), \mathsf{Enc}(B_b, R_{10} \oplus C_{(a\oplus 1)\wedge b})$$

Assumption 3.2.

$$(\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b, R_{10})) \overset{c}{\approx} (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b, R_{10} \oplus C_c \oplus \Delta))$$

Now, we make the following assumption to prove Theorem 3.2 by contradiction:

Assumption 3.3. There exists a successful adversary $\mathcal{A}$ that can break the security of $G^b_{\text{Sim-FreeXOR}}$, i.e.,

$$\mathsf{Adv}(\mathcal{A}, G^0_{\text{Sim-FreeXOR}}, G^1_{\text{Sim-FreeXOR}}) > \mathsf{negl}(\lambda)$$

However, since

$$\begin{aligned} G^0_{\text{Sim-FreeXOR}} & \overset{code}{\equiv} \mathcal{R} \to G^0_{\text{LIN-RK-KDM}} && \text{(Lemma 3.3)} \\ \text{and } G^1_{\text{Sim-FreeXOR}} & \overset{code}{\equiv} \mathcal{R} \to G^1_{\text{LIN-RK-KDM}} && \text{(Lemma 3.5)} \end{aligned}$$

we can rewrite Assumption 3.3 as

$$\mathsf{Adv}(\mathcal{A}, \mathcal{R} \to G^0_{\mathsf{LIN-RK-KDM}}, \mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}) > \mathsf{negl}(\lambda)$$

Package composition is associative, since the states of the packages are separated from each other in a state-separating proof [BO23, Section III(C)]. Therefore, we can rewrite Assumption 3.3 once again, as

$$\mathsf{Adv}(\mathcal{A} \to \mathcal{R}, G^0_{\mathsf{LIN-RK-KDM}}, G^1_{\mathsf{LIN-RK-KDM}}) > \mathsf{negl}(\lambda).$$

The above statement implies that there exists a PPT adversary $\mathcal{A} \to \mathcal{R}$ that achieves non-negligible advantage against the LIN-RK-KDM security game, contradicting Definition 3.1. Therefore, Assumption 3.3 is false. Hence, the advantage of any PPT adversary against $G^b_{\mathsf{Sim\text{-}FreeXOR}}$ is negligible, and Theorem 3.2 follows. $\square$

Chapter 4
Formalizing Free XOR in Domino

This chapter describes the formalization of the SSP introduced in the previous chapter using Domino. We first introduce the aspects of Domino related to the formalization, including its project structure and mechanisms used to represent cryptographic constructions and their security arguments. We then describe how the Free XOR construction was translated from its pen-and-paper description to Domino. While the reduction can be successfully checked by Domino, the equivalence relations could not be fully verified. Therefore, we discuss the invariants and randomness mappings required by these equivalences, as well as the limitations encountered during formalization.

4.1 Domino

Domino is a proof assistant for State-separating proofs (SSPs) that allows the user to validate a SSP using familiar synatax, and an underlying SMT solver. It uses very similar syntax and constructs as SSPs, such as packages, games and theorems. The explicit relation between these games is defined using invariant files, written in the SMT-LIB language. These games are then checked against different lemmata that are needed to fulfil the relation that the user wants to establish.

Domino is published as a rust package available on Github. The domino program/crate itself contains a compiler that has its own defined language that intentionally tries to be very similar the SSP syntax defined in [BDF+18].

4.1.1 Architecture

4.1.2 Packages

4.1.3 Compositions

4.1.4 Theorems

4.2 Setting up Domino

4.2.1 Dependencies

Since Domino is published as a Rust package, it requires a Rust toolchain, which can be installed using *rustup*. Rustup manages multiple Rust toolchains, and is the most straightforward way to getting set up on Linux. Rustup is typically installed using a bash script from the official website, but it is also available on most system package managers. It can also be built from source. Installing rustup automatically installs the Rust binary and cargo (the package manager), among other components. These can also be obtained without rustup, and the installation page explains the methods in more detail [Rus].

The second prerequisite to run Domino is a functioning CVC5 installation, and for the cvc5 executable to be added to PATH, so that Domino can invoke it while running. CVC5 can be built from source following the instructions in [CVC]. CVC5 is the SMT solver that checks whether the constraints given by our proof are satisfiable or not. This is done in practice by taking the negation of the condition we want, and checking for satisfiability of that negation. If it exists, the condition if false.

4.2.2 Project Structure

A Domino project directory requires the presence of an empty *ssp.toml* file. It allows SSP logic to be written into packages, games and theorem files. These are all placed under their own directories, i.e., folders called *packages*, *games*, and *theorem*.

Each of these folders can then contain one or more files. Domino checks packages and games for syntactic correctness even if they are not used in a theorem. This is intentional design, since when writing state separating proofs, the components (packages and games) are usually set up before the actual proof.

In older versions of Domino, theorems were called proofs, and the folder and file header for proofs were named as such.

4.3 Encoding the Free XOR SSP

4.3.1 From SSP to Domino

4.3.2 Packages

4.3.3 Compositions

4.3.4 Security Assumptions

4.3.5 Reductions

4.3.6 Equivalences

4.3.7 Verification Results

Chapter 5 Discussion

In this chapter, we discuss the advantages and challenges when using Domino compared to a pen-and-paper proof.

- Claim 4.6-4.8 from Applebaums proof that map to the game hops here
- underlying assumption and how this was programmed in domino
- adapting the lin rk kdm game to ssp
- domino verified the reduction game hop, but not the equivalence ones
- what does the acceptance of the reduction game hop mean
- why did it not accept them? - the information theoretic assumption where we swap the output wire label to the 1st ciphertext was difficult to model since statistical assumptions are hard. what would it take to model that? (another game hop with an assumption that those two games are computationally indistinguishable)

5.1 From Applebaum's proof to SSP

5.2 From SSP to Domino

- Advantages:
- Simplicity of the syntax and examples of mappings to SSP syntax
-
- Challenges:
- Real 0 bitstrings
- composing bitstrings to make larger bitstrings
- composing bits and bits or bits and bitstings into bitstrings
- indexing bitstrings by array index
- inline invoke commands for oracle calls
- index tuples by index to access their elements
- can't explicitly define random functions like enc so we have to use $enc(k, m, rand)$ and we needed to map the randomness between games explicitly (which is very

clunky and there's probably a better way to do this)
- all or nothing style of proofs where the oracle is treated as a single unit, so its hard to reason and debug about what is happening inside the oracle. its not very helpful in assisting thinking of assumptions needed to prove an equivalence.
- need a way to initialize Bits(n) tuples without using Tables
- randomness sampling is not aware of conditional blocks so we had to use very dirty randomness syntax
-

5.3 Verification results

Reformulate Applebaum's Free XOR proof as an SSP Achieved Encode the SSP in Domino Achieved Encode the security assumption Achieved Encode the reduction Achieved; verification succeeds Encode the game equivalences Partially achieved Verify the complete security argument Not achieved

5.4 Challenges and limitations

5.5 Future Work

- Proof of privacy when Point and Permute is added to this and special correctness is not required anymore
- Proof of privacy of Free XOR in a hash based model could be interesting
- Proof of privacy when we have multiple gates or a larger circuit
-

Chapter 6
Conclusion

References

[App16] B. Applebaum, "Garbling XOR gates "for free" in the standard model", *Journal of Cryptology*, vol. 29, no. 3, pp. 552–576, Jul. 2016.

[BCDS22] D. Basin, C. Cremers, et al., " Tamarin: Verification of Large-Scale, Real-World, Cryptographic Protocols ", *IEEE Security & Privacy*, vol. 20, no. 03, pp. 24–32, May 2022. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/MSEC.2022.3154689

[BDE+22] C. Brzuska, A. Delignat-Lavaud, et al., "Key-schedule security for the TLS 1.3 standard", in *Advances in Cryptology – ASIACRYPT 2022, Part I*, (Taipei, Taiwan, Dec. 5–9, 2022), S. Agrawal and D. Lin, Eds., ser. Lecture Notes in Computer Science, vol. 13791, Springer, Cham, Switzerland, 2022, pp. 621–650.

[BDF+18] C. Brzuska, A. Delignat-Lavaud, et al., "State separation for code-based game-playing proofs", in *Advances in Cryptology – ASIACRYPT 2018, Part III*, (Dec. 2–6, 2018), T. Peyrin and S. Galbraith, Eds., ser. Lecture Notes in Computer Science, vol. 11274, Brisbane, Queensland, Australia: Springer, Cham, Switzerland, Dec. 2018, pp. 222–249.

[BDG+14] G. Barthe, F. Dupressoir, et al., "EasyCrypt: A tutorial", in *Foundations of Security Analysis and Design VII: FOSAD 2012/2013 Tutorial Lectures*, A. Aldini, J. Lopez, and F. Martinelli, Eds., Cham: Springer International Publishing, 2014, pp. 146–166. last visited: Jul. 23, 2026. [Online]. Available: https://doi.org/10.1007/978-3-319-10082-1_6

[BERW] C. Brzuska, C. Egger, et al. "Domino". Code last validated against commit 6a61cdbb8193f321aad51890047caa52c93ff962, last visited: Jul. 13, 2026. [Online]. Available: https://github.com/domino-lang/domino

[BERW26] C. Brzuska, C. Egger, et al., "Domino: Computer-assisted game-hopping for large-scale protocols", manuscript, 2026.

[BHR12] M. Bellare, V. T. Hoang, and P. Rogaway, "Foundations of garbled circuits", in *ACM CCS 2012: 19th Conference on Computer and Communications Security*, (Oct. 16–18, 2012), T. Yu, G. Danezis, and V. D. Gligor, Eds., Raleigh, NC, USA: ACM Press, Oct. 2012, pp. 784–796.

[Bla16] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and proverif", *Found. Trends Priv. Secur.*, vol. 1, no. 1–2, pp. 1–135, Oct. 2016. [Online]. Available: https://doi.org/10.1561/3300000004

[Bla23] B. Blanchet, "CryptoVerif: a Computationally-Sound Security Protocol Verifier (Initial Version with Communications on Channels)", Inria Paris, Tech. Rep. RR-9525, Oct. 2023, p. 166. [Online]. Available: https://inria.hal.science/hal-04246199

[BMR90] D. Beaver, S. Micali, and P. Rogaway, "The round complexity of secure protocols (extended abstract)", in *22nd Annual ACM Symposium on Theory of Computing*, (Baltimore, MD, USA, May 14–16, 1990), ACM Press, 1990, pp. 503–513.

[BO23] C. Brzuska and S. Oechsner, "A state-separating proof for Yao's garbling scheme", in *CSF 2023: IEEE 36th Computer Security Foundations Symposium*, (Dubrovnik, Croatia, Jul. 10–14, 2023), IEEE Computer Society Press, 2023, pp. 137–152.

[BR06] M. Bellare and P. Rogaway, "The security of triple encryption and a framework for code-based game-playing proofs", in *Advances in Cryptology – EUROCRYPT 2006*, (May 28–Jun. 1, 2006), S. Vaudenay, Ed., ser. Lecture Notes in Computer Science, vol. 4004, St. Petersburg, Russia: Springer Berlin Heidelberg, Germany, May 2006, pp. 409–426.

[BR93] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols", in *ACM CCS 93: 1st Conference on Computer and Communications Security*, (Nov. 3–5, 1993), D. E. Denning, R. Pyle, et al., Eds., Fairfax, Virginia, USA: ACM Press, Nov. 1993, pp. 62–73.

[CKKZ12] S. G. Choi, J. Katz, et al., "On the security of the "free-XOR" technique", in *TCC 2012: 9th Theory of Cryptography Conference*, (Mar. 19–21, 2012), R. Cramer, Ed., ser. Lecture Notes in Computer Science, vol. 7194, Taormina, Sicily, Italy: Springer Berlin Heidelberg, Germany, Mar. 2012, pp. 39–53.

[CVC] CVC5. "Installation – cvc5 documentation", last visited: May 25, 2026. [Online]. Available: https://cvc5.github.io/docs/cvc5-1.0.0/installation/installation.html

[DY81] D. Dolev and A. C.-C. Yao, "On the security of public key protocols (extended abstract)", in *22nd Annual Symposium on Foundations of Computer Science*, (Oct. 28–30, 1981), Nashville, TN, USA: IEEE Computer Society Press, Oct. 1981, pp. 350–357.

[EGL82] S. Even, O. Goldreich, and A. Lempel, "A randomized protocol for signing contracts", in *Advances in Cryptology – CRYPTO'82*, D. Chaum, R. L. Rivest, and A. T. Sherman, Eds., Santa Barbara, CA, USA: Plenum Press, New York, USA, 1982, pp. 205–210.

[Eur16] European Parliament and the Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA relevance)*, May 2016. [Online]. Available: http://data.europa.eu/eli/reg/2016/679/oj

[Hal05] S. Halevi, *A plausible approach to computer-aided cryptographic proofs*, Cryptology ePrint Archive, Report 2005/181, 2005. [Online]. Available: https://eprint.iacr.org/2005/181

[HYLL24] N. Haloani, A. Yanai, et al., "COTI V2: Confidential Computing Ethereum Layer 2", COTI, Soda Labs, Tech. Rep., 2024. [Online]. Available: https://coti.io/files/coti_v2_whitepaper.pdf

[KS08] V. Kolesnikov and T. Schneider, "Improved garbled circuit: Free XOR gates and applications", in *Automata, Languages and Programming*, L. Aceto, I. Damgård, et al., Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 486–498.

[Lin17] Y. Lindell, "How to simulate it – a tutorial on the simulation proof technique", in *Tutorials on the Foundations of Cryptography*, Y. Lindell, Ed., Series Title: Information Security and Cryptography. Additional ISBNs: 978-3-319-57048-8, Cham: Springer International Publishing, 2017, pp. 277–346. last visited: May 21, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-319-57048-8_6

[LP09] Y. Lindell and B. Pinkas, "A proof of security of Yao's protocol for two-party computation", *Journal of Cryptology*, vol. 22, no. 2, pp. 161–188, Apr. 2009.

[LPR+21] T. Lepoint, S. Patel, et al., "Private join and compute from pir with default", 2021. [Online]. Available: https://eprint.iacr.org/2020/1011

[MNPS04] D. Malkhi, N. Nisan, et al., "Fairplay – a secure two-party computation system", in *Proceedings of the 13th conference on USENIX Security Symposium - Volume 13*, ser. SSYM'04, USA: USENIX Association, Aug. 13, 2004, p. 20. last visited: Jun. 25, 2026.

[NPS99] M. Naor, B. Pinkas, and R. Sumner, "Privacy preserving auctions and mechanism design", in *Proceedings of the 1st ACM Conference on Electronic Commerce*, ser. EC '99, Denver, Colorado, USA: Association for Computing Machinery, 1999, pp. 129–139. [Online]. Available: https://doi.org/10.1145/336992.337028

[Raj25] A. Rajabi, "Towards formally verifying the TLS 1.3 Key Schedule Security in SSBee", en, *Master thesis, Aalto University*, Jul. 2025. [Online]. Available: https://aaltodoc.aalto.fi/handle/123456789/138153

[RR21] M. Rosulek and L. Roy, "Three halves make a whole? Beating the half-gates lower bound for garbled circuits", in *Advances in Cryptology – CRYPTO 2021, Part I*, (Aug. 16–20, 2021), T. Malkin and C. Peikert, Eds., ser. Lecture Notes in Computer Science, vol. 12825, Virtual Event: Springer, Cham, Switzerland, Aug. 2021, pp. 94–124.

[Rus] Rust Foundation. "Other Installation Methods - Rust Forge", last visited: May 25, 2026. [Online]. Available: https://forge.rust-lang.org/infra/other-installation-methods.html

[Urb24] A. Urban, “Efficient delegated secure multiparty computation”, Theses, Institut Polytechnique de Paris, Dec. 2024. [Online]. Available: https://theses.hal.science/tel-04955820

[Yao86] A. C.-C. Yao, “How to generate and exchange secrets (extended abstract)”, in *27th Annual Symposium on Foundations of Computer Science*, (Toronto, Ontario, Canada, Oct. 27–29, 1986), IEEE Computer Society Press, 1986, pp. 162–167.

[ZRE15] S. Zahur, M. Rosulek, and D. Evans, “Two halves make a whole - reducing data transfer in garbled circuits using half gates”, in *Advances in Cryptology – EUROCRYPT 2015, Part II*, (Sofia, Bulgaria, Apr. 26–30, 2015), E. Oswald and M. Fischlin, Eds., ser. Lecture Notes in Computer Science, vol. 9057, Springer Berlin Heidelberg, Germany, 2015, pp. 220–250.

Appendix A
Ideal Game Transformations

This app describes the transformations to the ideal games in in order to clarify the transformations we make to claim code equivalence.

A.1 Game $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$

$\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$

State:

Δ : global circuit offset

$\mathsf{GARBLE}(f, x)$

$c \leftarrow f(x)$

if $\Delta = \perp$: // Always runs at the start once when Δ is unset

  $\Delta \leftarrow\$ \{0,1\}^\lambda$ // Sample a fresh Δ

$(a, b) \leftarrow x$

$A_a, B_b, C_c \leftarrow\$ \{0,1\}^\lambda$

$R_{00}, R_{01}, R_{10}, R_{11} \leftarrow\$ \{0,1\}^\lambda$

F : Table

$F[a][b] \leftarrow\$ (\mathsf{Enc}(A_a, R_{00}), \mathsf{Enc}(B_b, R_{00} \oplus C_c))$

$F[a][b \oplus 1] \leftarrow\$ (\mathsf{Enc}(A_a, R_{01}), \mathsf{Enc}(B_b \oplus \Delta, 0^\lambda))$

$\boxed{F[a \oplus 1][b] \leftarrow\$ (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b, R_{10}))}$

$F[a \oplus 1][b \oplus 1] \leftarrow\$ (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b \oplus \Delta, 0^\lambda))$

$X \leftarrow (A_a, B_b)$

$d \leftarrow \{C_c : c\}$

return (F, X, d)

Figure A.1: Code for $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$

Figure A.1 is a composition of the reduction $\mathcal{R}$ to the ideal LIN-RK-KDM game $G^1_{\mathsf{LIN-RK-KDM}}$.

A.2 Game $G^{1'}_{\mathsf{Sim\text{-}FreeXOR}}$

$G^{1'}_{\mathsf{Sim\text{-}FreeXOR}}$

State:

Δ : global circuit offset

$\mathsf{GARBLE}(f(x), \Phi(f))$

$c \leftarrow f(x)$ // This is 0 since we set a and b to 0

$\Delta \leftarrow\!\!\$ \{0,1\}^\lambda$ // this is equivalent to the

// if statement in the ideal version of $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$

$(a, b) \leftarrow (0, 0)$

$A_a, B_a, C_a \leftarrow\!\!\$ \{0,1\}^\lambda$

$R_{00}, R_{01}, R_{10}, R_{11} \leftarrow\!\!\$ \{0,1\}^\lambda$

F : Table

$F[a][b] \leftarrow\!\!\$ (\mathsf{Enc}(A_a, R_{00}), \mathsf{Enc}(B_b, R_{00} \oplus C_c))$

$F[a][b \oplus 1] \leftarrow\!\!\$ (\mathsf{Enc}(A_a, R_{01}), \mathsf{Enc}(B_b \oplus \Delta, 0^\lambda))$

$\boxed{F[a \oplus 1][b] \leftarrow\!\!\$ (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b, R_{10} \oplus C_c \oplus \Delta))}$

$F[a \oplus 1][b \oplus 1] \leftarrow\!\!\$ (\mathsf{Enc}(A_a \oplus \Delta, 0^\lambda), \mathsf{Enc}(B_b \oplus \Delta, 0^\lambda))$

$X \leftarrow (A_a, B_b)$

$d \leftarrow \{C_c : c\}$

return (F, X, d)

Figure A.2: Code for $G^{1'}_{\mathsf{Sim\text{-}FreeXOR}}$

Figure A.2 is a reformulation of the syntax of $G^{1'}_{\mathsf{LIN-RK-KDM}}$ (Figure 3.4) so that it is closer to the syntax of $\mathcal{R} \to G^1_{\mathsf{LIN-RK-KDM}}$.